# Exercise 4: Structs, pointers and references

**Discussion:** Tuesday, November 10

## Problem 4.1: Computing with complex numbers

Consider the C++ structure `Complex` as introduced in the lecture slide, representing complex numbers with real and imaginary parts, including operator overloading for addition. You can start from the source code file provided in the Moodle.

Extend the `Complex` struct with the following functionality:

a) **Polar Coordinate Member Variables & Constructors:** Add two additional member variables `double r` (magnitude) and `double phi` (phase angle in radians). Implement two constructor options:

- A constructor for Cartesian coordinates `(real, imag)` that automatically calculates the polar coordinates $r = \sqrt{\text{real}^2 + \text{imag}^2}$ and $\varphi = \arctan 2(\text{imag}, \text{real})$.
- A constructor/tagging approach for Polar coordinates `(r, phi, bool isPolar = true)` that automatically calculates the Cartesian coordinates $\text{real} = r \cdot \cos(\varphi)$ and $\text{imag} = r \cdot \sin(\varphi)$.

*(Hint: Use `std::sqrt`, `std::atan2`, `std::cos`, and `std::sin` from `<cmath>`)*

b) **Subtraction and Multiplication Operators:** Overload the `-` and `*` operators for two complex numbers, ensuring all member variables are properly updated.

c) **Printing Function:** Extend the `print()` function to display both Cartesian representation (`real + i imag`) and Polar representation (`r * e^(i * phi)`).

d) **Testing in `main()`:** Demonstrate your implementation inside `main()` by creating complex numbers using both Cartesian and polar constructors and displaying their operation results.

## Problem 4.2: Pointers and references

For this Exercise write your functions into `main.cpp`. Try out your code as usual in `main.cpp`.

a) Write a function `increment(int* address_to_number)` that increments the integer value at the given memory address. Next, write an overloaded version `increment(int& number)` that achieves the exact same result using a reference. Which of the two versions do you find easier to write and understand, and why?

b) Declare a `struct` named `Element` that stores an integer value and a pointer to the next `Element` named `next`. Then, create two instances, `first` and `second`. `first` should store the value `1` and point to `nullptr`, while `second` should store the value `2` and point to `first`. Finally, print the value of `first` by accessing it through `second`'s `next` pointer.

*Hint:* We did not yet talk about `nullptr`. The internet (or an AI chat bot) may help.